

Git Cheat Sheet

The essential Git commands & info every developer must know



Table of Content

What is Git ?.....	3
Definition.....	3
Key concepts of Git:.....	3
Stages of changes.....	4
Stages.....	4
Creating Snapshots.....	5
Initializing a repository.....	5
Staging files.....	5
Viewing the status.....	5
Committing the staged files.....	5
Skipping the staging area.....	5
Removing files.....	5
Renaming or moving files.....	5
Viewing the staged/unstaged changes.....	6
Viewing the history.....	6
Viewing a commit.....	6
Unstaging files (undoing git add).....	6
Discarding local changes.....	6
Restoring an earlier version of a file.....	7
Browsing History.....	8
Viewing the history.....	8
Filtering the history.....	8
Formatting the log output.....	8
Creating an alias.....	8
Viewing a commit.....	8
Comparing commits.....	8
Checking out a commit.....	9
Finding a bad commit.....	9
Finding contributors.....	9
Viewing the history of a file.....	9
Finding the author of lines.....	9
Tagging.....	9
Branching & Merging.....	10
Managing branches.....	10
Comparing branches.....	10
Stashing.....	10
Merging.....	10
Viewing the merged branches.....	11
Rebasing.....	11
Cherry picking.....	11
Merging types.....	11
Collaboration.....	12
Cloning a repository.....	12
Syncing with remotes.....	12
Sharing tags.....	12
Sharing branches.....	12
Managing remotes.....	12
Rewriting History.....	13

Undoing commits.....	13
Reverting commits.....	13
Recovering lost commits.....	13
Amending the last commit.....	13
Interactive rebasing.....	13
Amending the earlier commit.....	13

What is Git ?

Definition

Git is a decentralised version control system. It records all the changes made to our code over time in the special database called repository. Via this history we can see how changes were made, what changes were made and why/when changes were applied.

Key concepts of Git:

- Without Git we would store dedicated folder for the each project version like: v1.0.0, v1.0.1 and then merge them manually
- Allows to work together: to work simultaneously on the same feature development
- Git remembers the entire history of any changes made to the repository
- Git is immutable: so when you are changing the commit message or renaming the branch, then new entity with the same characteristics as the original gets created
- Git is a distributive **VCS** (version control system): every worker/team member has its own copy of the project on his machine with the entire history and changes, this way it's way more reliable than centralized VCS's like **TFS**
- **Head** is a pointer to the branch where we currently are located (currently checked-out branch)
- Git under the hood represents a bunch of sequential nodes(commits) linked with each other. Branches are represented as pointers to the specific nodes. And on top of the branch pointers there must stand the **Head** pointer
- Checking-out of some branch is no more than just moving of the **Head** pointer

Stages of changes

Stages

- Working directory
- Staging area
- Local repository (*index*)
- Remote repository



Transferring between changes

1. Initially created unit of work (file, folder, etc.) or update unit of work goes to the **working directory**.
2. After running “git add <match unit of work>” all the matched items will be added to the **staging area**. This stage isn’t connected with the neither local or remote repository, so the changes in this stage are needed as a preparation to be introduced to the repository, intermediate step.
3. To the **local repository** will go all the units of work ONLY from the **Staging area** by running “git commit -m "<commit message>"". Changes in this area stays independent from the states of the previous stages in the hierarchy (example: some files that previously were committed can be deleted in the working directory and after that to be recovered from the local repository). Local repository represents the same repository that many people can access remotely with one difference, that it’s not the actual repository from the remote, it’s your local copy of it that can be therefore integrated/pushed with the remote one.
4. Only after running “git push” your local repository instance is integrated with the **remote repository**. All the changes from your local instance are pushed and become the publicly accessible changes.

Creating Snapshots

Initializing a repository

git init

Staging files

git add file1.js # Stages a single file

git add file1.js file2.js # Stages multiple files

git add *.js # Stages with a pattern

git add . # Stages the current directory and all its content

Viewing the status

git status # Full status

git status -s # Short status

Committing the staged files

git commit -m "Message" # Commits with a one-line message

git commit # Opens the default editor to type a long message

Skipping the staging area

git commit -am "Message" # Works only when changes added were not untracked at all

Removing files

git rm file1.js # Removes from working directory and staging area

git rm --cached file1.js # Removes from staging area only

Renaming or moving files

git mv file1.js file1.txt

Viewing the staged/unstaged changes

git diff	# Shows unstaged changes
git diff --staged	# Shows staged changes
git diff --cached	# Same as the above

Viewing the history

git log	# Full history for the branch
git log --oneline	# Summary for the branch
git log --reverse	# Lists the commits from the oldest to the newest
git log --all	# Lists all the commits among all branches
git log --graph	# Builds graph with all the branches and commits
git log --all --oneline --graph	

Viewing a commit

git show 921a2ff	# Shows the given commit
git show HEAD	# Shows the last commit
git show HEAD~2	# Two steps before the last commit
git show HEAD:file.js	# Shows the version of file.js stored in the last commit

Unstaging files (undoing git add)

git restore --staged file.js	# Unstages file.js from the staging area to the working dir
------------------------------	---

Discarding local changes

git restore file.js	# Copies file.js from index to working directory
git restore file1.js file2.js	# Restores multiple files in working directory
git restore .	# Discards all local changes (except untracked files)
git clean -fd	# Removes all untracked files

Restoring an earlier version of a file

```
git restore --source=HEAD~2 file.js
```


Browsing History

Viewing the history

`git log --stat` # Shows the list of modified files
`git log --patch` # Shows the actual changes (patches)

Filtering the history

`git log -3` # Shows the last 3 entries
`git log --author="Mosh"`
`git log --before="2020-08-17" git`
`log --after="one week ago"`
`git log --grep="GUI"` # Commits with "GUI" in their message
`git log -S"GUI"` # Commits with "GUI" in their patches
`git log hash1..hash2` # Range of commits
`git log file.txt` # Commits that touched file.txt

Formatting the log output

`git log --pretty=format:"%an committed %H"`

Creating an alias

`git config --global alias.lg "log --oneline"`

Viewing a commit

`git show HEAD~2`
`git show HEAD~2:file1.txt` # Shows the version of file stored in this commit

Comparing commits

`git diff HEAD~2 HEAD` # Shows the changes between two commits
`git diff HEAD~2 HEAD file.txt` # Changes to file.txt only

Checking out a commit

git checkout dad47ed	# Checks out the given commit
git checkout master	# Checks out the master branch
git checkout -b feature-1	# Creates and checkouts branch
feature-1	

Finding a bad commit

git bisect start	
git bisect bad	# Marks the current commit as a bad commit
git bisect good ca49180	# Marks the given commit as a good commit
git bisect reset	# Terminates the bisect session

Finding contributors

git shortlog

Viewing the history of a file

git log file.txt	# Shows the commits that touched file.txt
git log --stat file.txt	# Shows statistics (the number of changes) for file.txt
git log --patch file.txt	# Shows the patches (changes) applied to file.txt

Finding the author of lines

git blame file.txt	# Shows the author of each line in file.txt
--------------------	---

Tagging

git tag v1.0	# Tags the last commit as v1.0
git tag v1.0 5e7a828	# Tags an earlier commit
git tag	# Lists all the tags
git tag -d v1.0	# Deletes the given tag

Branching & Merging

Managing branches

<code>git branch bugfix</code>	<code># Creates a new branch called bugfix</code>
<code>git checkout bugfix</code>	<code># Switches to the bugfix branch</code>
<code>git switch bugfix</code>	<code># Same as the above</code>
<code>git switch -C bugfix</code>	<code># Creates and switches</code>
<code>git branch -d bugfix</code>	<code># Deletes the bugfix branch</code>

Comparing branches

<code>git log master..bugfix</code>	<code># Lists the commits in the bugfix branch not in master</code>
<code>git diff master..bugfix</code>	<code># Shows the summary of changes</code>

Stashing

<code>git stash push -m "New tax rules"</code>	<code># Creates a new stash</code>
<code>git stash list</code>	<code># Lists all the stashes</code>
<code>git stash show stash@{1}</code>	<code># Shows the given stash</code>
<code>git stash show 1</code>	<code># shortcut for stash@{1}</code>
<code>git stash apply 1</code>	<code># Applies the given stash to the working dir</code>
<code>git stash drop 1</code>	<code># Deletes the given stash</code>
<code>git stash clear</code>	<code># Deletes all the stashes</code>

Merging

<code>git merge bugfix</code>	<code># Merges the bugfix branch into the current branch</code>
<code>git merge --no-ff bugfix</code>	<code># Creates a merge commit even if FF is possible</code>
<code>git merge --squash bugfix</code>	<code># Performs a squash merge</code>
<code>git merge --abort</code>	<code># Aborts the merge</code>
<code>git merge --ff-only bugfix</code>	<code># Allows only fast-forward merge, otherwise discard</code>

Viewing the merged branches

`git branch --merged` # Shows the merged branches

`git branch --no-merged` # Shows the unmerged branches

Rebasing

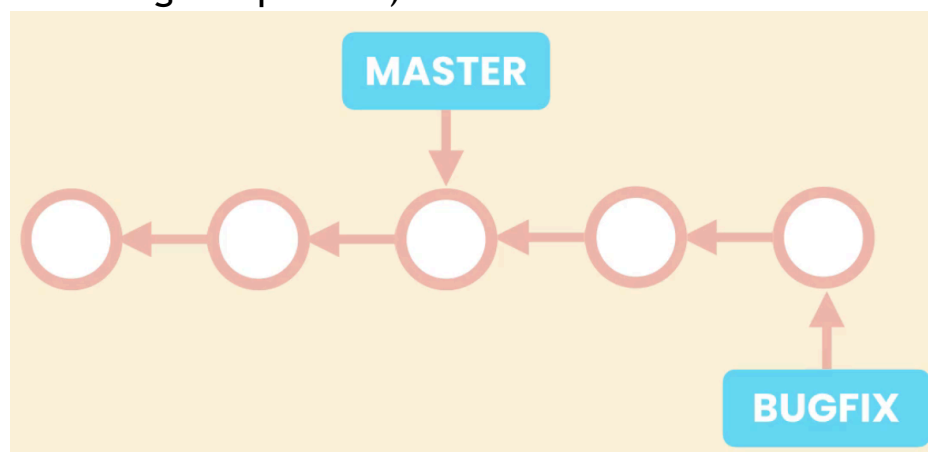
`git rebase master` # Changes the base of the current branch

Cherry picking

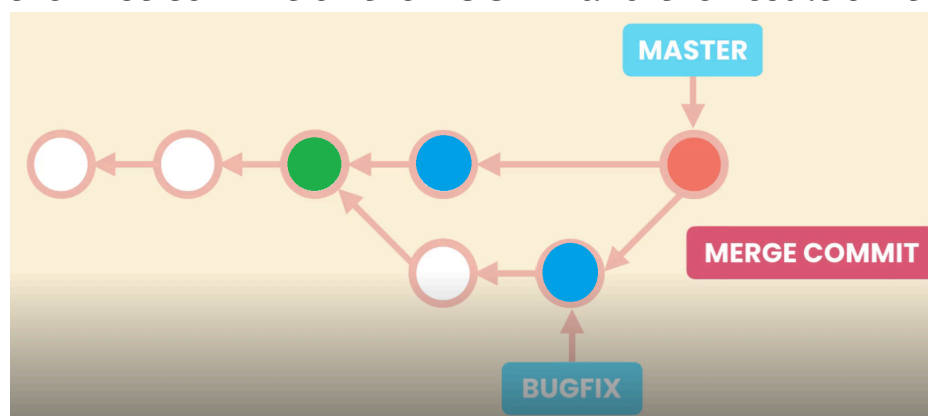
`git cherry-pick dad47ed` # Applies the given commit on the current branch

Merging types

- Fast-forward = simply move forward the **MASTER** pointer to the **BUGFIX** (merge when the last commit of the **MASTER** is the first commit of the **BUGFIX** and the result of this operation is moving the pointer)



- 3-way merge = movement further of the **MASTER** pointer is not possible to the **BUGFIX** without losing the commits from the **BUGFIX** (merge when the last commit of **MASTER** is NOT the first commit of the **BUGFIX** and the result of it is the new commit called *merge commit*)



(3-way merge since is based on the green circle - last MASTER commit in the BUGFIX branch and on the blue circles that are last commits in MASTER and BUGFIX)

Collaboration

Cloning a repository

git clone url

Syncing with remotes

git fetch origin master

Fetches master from origin

git fetch origin

Fetches all objects from origin

git fetch

Shortcut for “git fetch origin”

git pull

Fetch + merge

git push origin master

Pushes master to origin

git push

Shortcut for “git push origin master”

Sharing tags

git push origin v1.0

Pushes tag v1.0 to origin

git push origin --delete v1.0

Sharing branches

git branch -r

Shows remote tracking branches

git branch -vv

Shows local & remote tracking branches

git push -u origin bugfix

Pushes bugfix to origin

git push -d origin bugfix

Removes bugfix from origin

Managing remotes

git remote

Shows remote repos

git remote add upstream url

Adds a new remote called upstream

git remote rm upstream

Removes upstream

Rewriting History

Undoing commits

`git reset --soft HEAD^` # Removes the last commit, keeps changed staged

`git reset --mixed HEAD^` # Unstages the changes as well

`git reset --hard HEAD^` # Discards local changes

`git reset --hard ORIG_HEAD` # Reset to the moment before merge/rebase was performed

Reverting commits

`git revert 72856ea` # Reverts the given commit

`git revert HEAD~3..` # Reverts the last three commits

`git revert --no-commit HEAD~3..` # Creates only 1 revert message

Recovering lost commits

`git reflog` # Shows the history of HEAD

`git reflog show bugfix` # Shows the history of bugfix pointer

Amending the last commit

`git commit --amend` # Allows to change the commit message and to add additional changes if the staging are at the moment of amending isn't empty

Interactive rebasing

`git rebase -i HEAD~5`

Amending the earlier commit

1. `git rebase -i HEAD~3`
2. Instead of “pick” option to the certain commit just pass “e” or “edit” in the opened window for the interactive rebase